Complete Java Streams Mastery Guide for SDETs

Table of Contents

---

1. Introduction to Streams

Java Streams were introduced in Java 8.

Streams help us process collections of data in a cleaner and smarter way.

Without Streams:

- more loops
- more temporary variables
- more boilerplate code

With Streams:

- cleaner code
- readable code
- functional programming style
- easier data processing

---

2. Why Streams are Important for SDETs

Streams are heavily used in:

SDET Use Case	Example
Filtering test data	Get failed users
API response validation	Extract IDs
Selenium data validation	Validate dropdown values
Database validation	Compare records
Log analysis	Find ERROR logs
Reporting	Count passed tests
Excel data processing	Read and filter rows
JSON processing	Extract nested values

---

3. Traditional Loop vs Streams

Traditional Way

```
List<String> names = Arrays.asList("Amit", "John", "Alex");
```

```
for(String name : names) {
    if(name.startsWith("A")) {
        System.out.println(name);
    }
}
```

Stream Way

```
names.stream()
     .filter(name -> name.startsWith("A"))
     .forEach(System.out::println);
```

Explanation

Stream version is:

- shorter
- cleaner
- easier to understand

---

4. What is a Stream?

A Stream is:

```
A sequence of data elements processed one by one.
```

Important:

- Stream does NOT store data.
- Stream processes data.

Think of Stream like:

```
Factory Assembly Line
```

Data moves step-by-step through operations.

---

5. How Streams Work Internally

Streams work in 3 stages:

```
Source → Intermediate Operations → Terminal Operation
```

Example:

```
names.stream()
     .filter(name -> name.startsWith("A"))
     .map(String::toUpperCase)
     .forEach(System.out::println);
```

Breakdown

Stage	Purpose
Source	Collection
filter()	Intermediate
map()	Intermediate
forEach()	Terminal

---

6. Creating Streams

6.1 From List

Example 1

```
List<String> names = Arrays.asList("Amit", "John", "Alex");

Stream<String> stream = names.stream();
```

Example 2

```
List<Integer> numbers = Arrays.asList(1,2,3,4,5);

numbers.stream().forEach(System.out::println);
```

6.2 From Array

Example 1

```
String[] arr = {"A", "B", "C"};

Arrays.stream(arr)
      .forEach(System.out::println);
```

Example 2

```
int[] nums = {1,2,3};

Arrays.stream(nums)
      .forEach(System.out::println);
```

6.3 Using Stream.of()

Example 1

```
Stream.of("Chrome", "Firefox", "Edge")
      .forEach(System.out::println);
```

Example 2

```
Stream.of(10,20,30)
      .forEach(System.out::println);
```

---

7. Stream Pipeline

A Stream pipeline means chaining operations together.

Example 1

```
List<Integer> nums = Arrays.asList(1,2,3,4,5);

nums.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * 10)
    .forEach(System.out::println);
```

Explanation

Step 1:

- filter even numbers

Step 2:

- multiply by 10

Step 3:

- print result

Example 2

```
List<String> browsers = Arrays.asList("chrome", "firefox", "edge");

browsers.stream()
        .map(String::toUpperCase)
        .forEach(System.out::println);
```

---

8. Intermediate vs Terminal Operations

Intermediate Operations

These return another Stream.

Examples:

- filter()
- map()
- sorted()

Terminal Operations

These produce final result.

Examples:

- collect()
- forEach()
- count()

Important Concept

Without terminal operation:

```
Stream will NOT execute.
```

---

9. filter()

Used to filter data.

Syntax

```
filter(condition)
```

Example 1 — Filter Even Numbers

```
List<Integer> nums = Arrays.asList(1,2,3,4,5,6);

nums.stream()
    .filter(n -> n % 2 == 0)
    .forEach(System.out::println);
```

Output:

```
2
4
6
```

Example 2 — Filter Failed Test Cases

```
List<String> status = Arrays.asList("PASS", "FAIL", "PASS", "FAIL");

status.stream()
      .filter(s -> s.equals("FAIL"))
      .forEach(System.out::println);
```

Explanation

filter() keeps only matching elements.

---

10. map()

Used to transform data.

Syntax

```
map(transformation)
```

Example 1 — Convert to Uppercase

```
List<String> names = Arrays.asList("amit", "john");

names.stream()
     .map(String::toUpperCase)
     .forEach(System.out::println);
```

Example 2 — Multiply Numbers

```
List<Integer> nums = Arrays.asList(1,2,3);

nums.stream()
    .map(n -> n * 10)
    .forEach(System.out::println);
```

Explanation

map() transforms each element.

---

11. collect()

Used to collect Stream result.

Example 1 — Collect to List

```
List<String> result = names.stream()
                           .filter(name -> name.startsWith("A"))
                           .collect(Collectors.toList());
```

Example 2 — Collect Uppercase Names

```
List<String> result = names.stream()
                           .map(String::toUpperCase)
                           .collect(Collectors.toList());
```

Explanation

collect() converts stream result into:

- List
- Set
- Map

---

12. forEach()

Used to iterate elements.

Example 1

```
names.stream()
     .forEach(System.out::println);
```

Example 2

```
nums.stream()
    .forEach(n -> System.out.println(n * 2));
```

---

13. sorted()

Used for sorting.

Example 1 — Ascending Order

```
List<Integer> nums = Arrays.asList(5,1,3,2);

nums.stream()
    .sorted()
    .forEach(System.out::println);
```

Example 2 — Sort Names

```
names.stream()
     .sorted()
     .forEach(System.out::println);
```

---

14. distinct()

Removes duplicates.

Example 1

```
List<Integer> nums = Arrays.asList(1,1,2,2,3,3);

nums.stream()
    .distinct()
    .forEach(System.out::println);
```

Example 2 — Unique Browser Names

```
List<String> browsers = Arrays.asList("chrome", "chrome", "edge");

browsers.stream()
        .distinct()
        .forEach(System.out::println);
```

---

15. limit()

Limits result count.

Example 1

```
nums.stream()
    .limit(3)
    .forEach(System.out::println);
```

Example 2 — Top Failed Tests

```
failedTests.stream()
           .limit(5)
           .forEach(System.out::println);
```

---

16. skip()

Skips elements.

Example 1

```
nums.stream()
    .skip(2)
    .forEach(System.out::println);
```

Example 2

```
logs.stream()
    .skip(1)
    .forEach(System.out::println);
```

---

17. count()

Counts elements.

Example 1

```
long count = nums.stream()
                 .count();
```

Example 2 — Count Failed Tests

```
long failed = status.stream()
                    .filter(s -> s.equals("FAIL"))
                    .count();
```

---

18. anyMatch(), allMatch(), noneMatch()

Used for validations.

18.1 anyMatch()

Checks if ANY element matches.

Example 1

```
boolean result = nums.stream()
                     .anyMatch(n -> n > 10);
```

Example 2 — Failed Test Exists

```
boolean failed = status.stream()
                       .anyMatch(s -> s.equals("FAIL"));
```

18.2 allMatch()

Checks if ALL elements match.

Example 1

```
boolean result = nums.stream()
                     .allMatch(n -> n > 0);
```

Example 2 — All Tests Passed

```
boolean passed = status.stream()
                       .allMatch(s -> s.equals("PASS"));
```

18.3 noneMatch()

Checks if NO element matches.

Example 1

```
boolean result = nums.stream()
                     .noneMatch(n -> n < 0);
```

Example 2

```
boolean noFail = status.stream()
                       .noneMatch(s -> s.equals("FAIL"));
```

---

19. findFirst() and findAny()

19.1 findFirst()

Returns first element.

Example 1

```
Optional<Integer> first = nums.stream()
                              .findFirst();
```

Example 2 — First Failed Test

```
Optional<String> fail = status.stream()
                              .filter(s -> s.equals("FAIL"))
                              .findFirst();
```

19.2 findAny()

Returns any matching element.

Example 1

```
Optional<Integer> any = nums.stream()
                            .findAny();
```

Example 2

```
Optional<String> browser = browsers.stream()
                                   .findAny();
```

---

20. reduce()

Used to combine elements into single result.

Example 1 — Sum Numbers

```
int sum = nums.stream()
              .reduce(0, Integer::sum);
```

Example 2 — Concatenate Strings

```
String result = names.stream()
                     .reduce("", (a,b) -> a + b);
```

Explanation

reduce() reduces multiple values into one.

---

21. flatMap()

Used to flatten nested collections.

Example 1

```
List<List<String>> data = Arrays.asList(
    Arrays.asList("A", "B"),
    Arrays.asList("C", "D")
);

data.stream()
    .flatMap(List::stream)
    .forEach(System.out::println);
```

Example 2 — Multiple Test Data Sheets

```
List<List<Integer>> sheets = Arrays.asList(
    Arrays.asList(1,2),
    Arrays.asList(3,4)
);

sheets.stream()
      .flatMap(List::stream)
      .forEach(System.out::println);
```

---

22. Stream with Objects

Very important for SDETs.

Sample Class

```
class Employee {
    String name;
    int salary;

    Employee(String name, int salary) {
        this.name = name;
        this.salary = salary;
    }
}
```

Example 1 — Filter Employees

```
employees.stream()
         .filter(emp -> emp.salary > 50000)
         .forEach(emp -> System.out.println(emp.name));
```

Example 2 — Extract Names

```
employees.stream()
         .map(emp -> emp.name)
         .forEach(System.out::println);
```

---

23. Stream with Test Data

Example 1 — Filter Failed Users

```
users.stream()
     .filter(user -> user.getStatus().equals("FAIL"))
     .forEach(System.out::println);
```

Example 2 — Extract Emails

```
users.stream()
     .map(User::getEmail)
     .forEach(System.out::println);
```

---

24. Stream with API Responses

Example 1 — Extract User IDs

```
ids.stream()
   .filter(id -> id > 100)
   .forEach(System.out::println);
```

Example 2 — Validate Status Codes

```
statusCodes.stream()
           .allMatch(code -> code == 200);
```

---

25. Stream with Selenium Data

Example 1 — Validate Dropdown Values

```
options.stream()
       .map(WebElement::getText)
       .forEach(System.out::println);
```

Example 2 — Find Specific Option

```
boolean found = options.stream()
                       .map(WebElement::getText)
                       .anyMatch(text -> text.equals("India"));
```

26. Stream with Database Data

Example 1 — Filter Active Users

```
users.stream()
     .filter(user -> user.isActive())
     .forEach(System.out::println);
```

Example 2 — Count Records

```
long total = users.stream()
                  .count();
```

27. Stream with Files and Logs

Example 1 — Find ERROR Logs

```
logs.stream()
    .filter(log -> log.contains("ERROR"))
    .forEach(System.out::println);
```

Example 2 — Count WARN Logs

```
long warnCount = logs.stream()
                     .filter(log -> log.contains("WARN"))
                     .count();
```

28. Grouping and Partitioning

28.1 groupingBy()

Groups elements.

Example 1

```
Map<String, List<Employee>> grouped =
employees.stream()
         .collect(Collectors.groupingBy(emp -> emp.department));
```

Example 2 — Group Test Results

```
Map<String, List<TestResult>> grouped =
results.stream()
       .collect(Collectors.groupingBy(TestResult::getStatus));
```

28.2 partitioningBy()

Partitions into true/false.

Example 1

```
Map<Boolean, List<Integer>> result =
nums.stream()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));
```

Example 2 — Pass/Fail Split

```
Map<Boolean, List<TestResult>> data =
results.stream()
       .collect(Collectors.partitioningBy(r -> r.isPassed()));
```

---

29. Collectors Utility Methods

29.1 joining()

Example 1

```
String joined = names.stream()
                     .collect(Collectors.joining(","));
```

Example 2

```
String report = status.stream()
                      .collect(Collectors.joining(" | "));
```

29.2 toSet()

Example 1

```
Set<Integer> set = nums.stream()
                       .collect(Collectors.toSet());
```

Example 2

```
Set<String> browsers = browserList.stream()
                                  .collect(Collectors.toSet());
```

---

30. Optional Class

Optional helps avoid NullPointerException.

Example 1

```
Optional<String> first = names.stream()
                              .findFirst();
```

Example 2

```
first.ifPresent(System.out::println);
```

---

31. Parallel Streams

Parallel streams process data in parallel.

Example 1

```
nums.parallelStream()
    .forEach(System.out::println);
```

Example 2 — Faster Large Data Processing

```
logs.parallelStream()
    .filter(log -> log.contains("ERROR"))
    .count();
```

Important Warning

Avoid parallel streams when:

- order matters
- shared data exists
- debugging is difficult

---

32. Method References

Cleaner alternative to lambdas.

Example 1

```
names.forEach(System.out::println);
```

Example 2

```
names.stream()
     .map(String::toUpperCase)
     .forEach(System.out::println);
```

---

33. Lambda Expressions

Lambdas are anonymous functions.

Example 1

```
n -> n * 2
```

Example 2

```
name -> name.startsWith("A")
```

---

34. Stream Debugging

Using peek()

Example 1

```
nums.stream()
    .peek(System.out::println)
    .filter(n -> n > 2)
    .forEach(System.out::println);
```

Example 2

```
names.stream()
     .peek(name -> System.out.println("Before: " + name))
     .map(String::toUpperCase)
     .forEach(System.out::println);
```

---

35. Common Beginner Mistakes

Mistake	Problem
Forgetting terminal operation	Stream won't execute
Reusing consumed stream	IllegalStateException
Overusing streams	Reduced readability
Using parallel stream everywhere	Performance issues
Complex lambdas	Hard maintenance

---

36. Stream Performance Tips

Tip 1

Use streams for readability.

Tip 2

Avoid unnecessary stream chaining.

Tip 3

Use parallel streams carefully.

Tip 4

Prefer method references when possible.

---

37. Real SDET Scenarios

Scenario	Stream Usage
Failed test filtering	filter()
API response transformation	map()
Duplicate data removal	distinct()
Test report generation	collect()
Dropdown validation	anyMatch()
Log validation	filter() + count()
Data aggregation	reduce()
Nested test data	flatMap()

---

38. Interview-Oriented Stream Questions

Most asked topics:

1. Difference between map() and flatMap()
2. Difference between intermediate and terminal operations
3. How Streams work internally
4. Difference between findFirst() and findAny()
5. What is lazy evaluation?
6. Difference between Collection and Stream
7. When to use parallelStream()
8. Difference between filter() and map()
9. What is reduce()?
10. What is Optional?

---

39. Streams Cheat Sheet

Method	Purpose
filter()	filtering
map()	transformation
collect()	collect result
sorted()	sorting
distinct()	remove duplicates
limit()	limit elements
skip()	skip elements
count()	count elements
reduce()	aggregate result
flatMap()	flatten collections
anyMatch()	any condition
allMatch()	all condition
noneMatch()	no condition
findFirst()	first element
findAny()	any element

---

40. Final Learning Roadmap

Beginner Level

Learn:

- Stream basics
- filter()
- map()
- collect()
- forEach()

Practice:

- numbers
- strings
- simple validations

Intermediate Level

Learn:

- sorted()
- distinct()
- reduce()
- Optional
- groupingBy()

Practice:

- API data
- test reports
- logs
- Selenium data

Advanced Level

Learn:

- flatMap()
- parallel streams
- advanced collectors
- complex object streams

Practice:

- nested test data
- DB validations
- large file processing
- complex reporting

Final Advice for SDETs

Streams become easy when:

- you practice daily
- you solve real automation problems
- you think step-by-step
- you understand data flow

Do NOT memorize stream syntax.

Instead understand:

```
How data moves through the pipeline.
```

That is the real key to mastering Streams in Java.